Exercises for
Introduction to Machine Learning
Linda-Sophie Schneider, Johannes Enk, Gia Long Nguyen,
Fengyu Han, Yangbin Peng, Lukas Huettner
Task 0: deadline: 20. May 2025

Pattern Recognition Lab

Warmup: Getting started

As the first exercise for this lecture, you will have to install and set up some software that will be required throughout the semester. Note that, in case of issues, try to first search for a solution on Google, as it might take us some time to respond to you[1].

Exercise 1 Python & IDE

For the exercises, we are using Python 3. Please install Python 3.10 or a newer version. If you use Windows or macOS, you can download Python from: `https://www.python.org/downloads/`. During the installation process, please make sure to check the "Add Python X.X to your PATH" option. If you use Linux, use your usual repository manager. To test whether Python is installed correctly, open a terminal and type `python --version`. If it returns the version number you just installed, everything is okay.

You can either use a simple text editor like Geany and the command line for programming, or you can install an Integrated Development Environment (IDE). As an IDE, we suggest using PyCharm. You can download the free community edition from `https://www.jetbrains.com/pycharm/download/#section=windows`. During the installation process, make sure to select "Add launchers dir to the PATH". For a more detailed explanation on how to install and set up PyCharm correctly, please see the *[MacOS] or [Windows] Installing PyCharm Professional and License Activation.pdf* on StudOn.

Exercise 2 Package Manager and Libraries

We suggest using a virtual environment to avoid cluttering your system.

If you use Windows or macOS, to set up a virtual environment in PyCharm, go to File > Settings > Project > Project Interpreter, click the settings icon at the top right. Then, choose Virtualenv Environment on the left side and select New environment. The base interpreter should be your installed Python version.

If you use Linux, then we recommend using `virtualenv` for creating virtual environments:

```
virtualenv -p python3 venv3
```

where `venv3` is the folder that will contain the virtual environment (any other name is also acceptable). You can use the same virtual environment for the entire semester; there is no need to create a new one for each exercise. You can activate an environment at any time with

```
source venv3/bin/activate
```

[1] There are more students than tutors. Many more.

As a package manager, we suggest using pip, which is included in the regular installation of Python. To install a library, you just need to type `pip install library_name` in the command line. All installed libraries can be viewed by typing `pip list`. PyCharm has an integrated command line, which you can find at the bottom of the window under the index tab `Terminal`.

You will need the libraries `numpy`, `opencv` and `matplotlib` for our exercises. Please install them.

Exercise 3 Image Processing Framework

For the first exercise, you will be tasked with writing a small and simple image processing library that supports a few functions. Please download the archive that we provided on StudOn. It contains two files: *exercise_0.py*, which contains a framework for the exercise, and *test.py*, which contains unit tests to automatically verify if your code works well.

We want to implement the *ImageProcessor class*. This class should offer a few different capabilities. Functions to show and save images are already provided. You should use them to verify your implementations.

If not stated otherwise, please avoid using external libraries or loops. Instead, try to use (conditional) indexing to perform the tasks.

For the first task, please finish the *constructor*. The *constructor* receives the path to an image and the desired color scheme (either BGR, RGB, or Gray). Load the image using CV2 and ensure that the saved image has the correct color scheme at the end.

Next, please implement the *convert_color* function. The function only works on color images (BGR and RGB) and should turn an RGB image into a BGR image. Verify your results by plotting the changed image.

Sometimes in image processing, it can be necessary to clip image intensities to certain values in order to mitigate the impact of, for example, very bright artifacts. Therefore, implement the *clip* function that receives a minimum and maximum value and clips all color intensities or pixel values based on the provided minimum and maximum.

When training convolutional neural networks on images, data augmentation is often performed to bring variability into the data. Those augmentations include flipping, rotating, cropping, and many more. Your task will be to implement a *flip* function that receives a parameter and based on that, either flips the image vertically (0), horizontally (1), or both ways (2). Again, please do not use any external libraries.

Please verify all your results graphically by plotting the images and running the tests.

```
$ python3 test.py
...
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

Important note: When we write the unit tests, we think about what can possibly go wrong in your code. However, teaching has taught us that students can be very ingenious in finding new ways to make mistakes. Passing the unit tests means only that it is very likely that the solution is correct – not more. This is why, in addition to submitting your solutions to StudOn, you will also have to present them to a tutor.

Submit the Python files to show everything is working correctly. **Never** include your virtual environment in your submission!